

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в интеллектуальных системах»

**Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях:
сравнение активационных функций»**

Выполнил:

Студент 4 курса

Группы ИИ-26

Яшков П. А.

Проверил:

Никитин С. К.

Брест 2026

Цель работы:

На основе многослойного персептрона архитектуры 2-2-1 из лабораторной работы №2 исследовать влияние активационной функции ReLU на скрытом слое. Провести сравнительный анализ сети с Сигмоидой и ReLU по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Ход работы:

В лабораторной работе используется вариант 10: $c_0 = -3$, $c_1 = -8$. Для обеих конфигураций применяется функция потерь Binary Cross-Entropy, на выходном слое используется Сигмоида, а на скрытом слое сравниваются Сигмоида и ReLU.

Вариант 10 ($c_0 = -3$, $c_1 = -8$)

A	B	$A \oplus B$
-3	-3	-3
-3	-8	-8
-8	-3	-8
-8	-8	-3

Основные изменения по сравнению с лабораторной работой №2:

1. В базовой конфигурации на скрытом слое используется Сигмоида.
2. В новой конфигурации на скрытом слое реализована ReLU, а на выходе сохранена Сигмоида.
3. Для ReLU реализована производная, равная 1 при положительном входе и 0 при неположительном.
4. Для обеих конфигураций используется BCE и одинаковый критерий остановки $E_s \leq 0,03$.
5. Выполняется по 5 запусков с разными seed и фиксируются случаи появления «мёртвых» ReLU-нейронов.

Код программы:

```
import sys
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt

# =====
# Лабораторная работа №3
# Вариант 10:  $c_0 = -3$ ,  $c_1 = -8$ 
# MLP 2-2-1 для XOR
# Сравнение Сигмоиды и ReLU на скрытом слое
# Функция потерь: Binary Cross-Entropy (BCE)
# =====

C0 = -3.0
C1 = -8.0

X_REAL = np.array([
    [-3.0, -3.0],
    [-3.0, -8.0],
    [-8.0, -3.0],
    [-8.0, -8.0]
], dtype=float)

Y_REAL = np.array([
    [-3.0],
    [-8.0],
    [-8.0],
    [-8.0],
])
```

```

], dtype=float)

# Параметры обучения
LEARNING_RATE = 0.1
EE = 0.03
MAX_EPOCHS = 20000
SEEDS = [1, 4, 5, 6, 7]
SHUFFLE_SEED = 42
INIT_SCALE = 0.3

# Папка для автоматически сохраняемых графиков
FIG_DIR = Path("lab3_variant10_figures")

# =====
# 1. Нормализация / денормализация
# =====

def normalize(value, c0=C0, c1=C1):
    """c0 -> 0, c1 -> 1."""
    return (value - c0) / (c1 - c0)

def denormalize(value, c0=C0, c1=C1):
    """Обратное преобразование из нормализованной шкалы."""
    return c0 + value * (c1 - c0)

X = normalize(X_REAL)
Y = normalize(Y_REAL)

# =====
# 2. Функции активации и потерь
# =====

def sigmoid(z):
    z = np.clip(z, -60.0, 60.0)
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_derivative_from_output(y):
    return y * (1.0 - y)

def relu(z):
    return np.maximum(0.0, z)

def relu_derivative(z):
    return (z > 0.0).astype(float)

def bce_loss(y_true, y_pred):
    eps = 1e-12
    y_pred = np.clip(y_pred, eps, 1.0 - eps)
    return -np.sum(
        y_true * np.log(y_pred)
        + (1.0 - y_true) * np.log(1.0 - y_pred)
    )

def classification_accuracy(y_true, y_pred):
    true_class = (y_true >= 0.5).astype(int)
    pred_class = (y_pred >= 0.5).astype(int)
    return float(np.mean(true_class == pred_class))

def mae_real(y_true_real, y_pred_norm):
    y_pred_real = denormalize(y_pred_norm)
    return float(np.mean(np.abs(y_true_real - y_pred_real)))

# =====
# 3. MLP 2-2-1
# =====

class MLP221:
    def __init__(self, hidden_activation="sigmoid", seed=1):
        if hidden_activation not in ("sigmoid", "relu"):
            raise ValueError("hidden_activation должен быть 'sigmoid' или 'relu'")

```

```

self.hidden_activation = hidden_activation
rng = np.random.RandomState(seed)

# Малые случайные веса.
self.W1 = rng.uniform(-INIT_SCALE, INIT_SCALE, size=(2, 2))

# Для ReLU небольшое положительное случайное смещение уменьшает
# вероятность мгновенного выключения обоих скрытых нейронов,
# но не исключает появления "мёртвого" нейрона в процессе обучения.
if hidden_activation == "relu":
    self.b1 = rng.uniform(0.01, INIT_SCALE, size=(1, 2))
else:
    self.b1 = rng.uniform(-INIT_SCALE, INIT_SCALE, size=(1, 2))

self.W2 = rng.uniform(-INIT_SCALE, INIT_SCALE, size=(2, 1))
self.b2 = rng.uniform(-INIT_SCALE, INIT_SCALE, size=(1, 1))

def hidden_forward(self, z1):
    if self.hidden_activation == "sigmoid":
        return sigmoid(z1)
    return relu(z1)

def hidden_derivative(self, z1, h):
    if self.hidden_activation == "sigmoid":
        return sigmoid_derivative_from_output(h)
    return relu_derivative(z1)

def forward(self, x):
    z1 = x @ self.W1 + self.b1
    h = self.hidden_forward(z1)

    z2 = h @ self.W2 + self.b2
    y_hat = sigmoid(z2)

    return z1, h, y_hat

def predict(self, x):
    return self.forward(x)[2]

def dead_neurons(self, x_train):
    """Возвращает логический массив: какие ReLU-нейроны мёртвые."""
    if self.hidden_activation != "relu":
        return np.array([False, False])

    z1 = x_train @ self.W1 + self.b1
    h = relu(z1)
    return np.all(h <= 1e-12, axis=0)

def fit(
    self,
    x_train,
    y_train,
    learning_rate=LEARNING_RATE,
    ee=EE,
    max_epochs=MAX_EPOCHS,
    shuffle_seed=SHUFFLE_SEED
):
    rng = np.random.RandomState(shuffle_seed)
    history = []
    converged = False

    for epoch in range(1, max_epochs + 1):
        indices = rng.permutation(len(x_train))

        # Онлайн-режим: обновление после каждого примера.
        for i in indices:
            x = x_train[i:i + 1]
            target = y_train[i:i + 1]

            z1, h, y_hat = self.forward(x)

            # BCE + sigmoid на выходе:
            # dE/dz2 = y_hat - target
            delta2 = y_hat - target

            # Ошибка скрытого слоя зависит от его активации.
            delta1 = (
                delta2 @ self.W2.T
                * self.hidden_derivative(z1, h)
            )

            self.W2 -= learning_rate * (h.T @ delta2)
            self.b2 -= learning_rate * delta2

            self.W1 -= learning_rate * (x.T @ delta1)
            self.b1 -= learning_rate * delta1

        predictions = self.predict(x_train)
        error = float(bce_loss(y_train, predictions))
        history.append(error)

```

```

        if error <= ee:
            converged = True
            break

    return {
        "epochs": epoch,
        "final_error": error,
        "history": history,
        "converged": converged
    }

# =====
# 4. Один запуск
# =====

def run_one(hidden_activation, seed):
    model = MLP221(hidden_activation=hidden_activation, seed=seed)

    training = model.fit(X, Y)
    pred_norm = model.predict(X)
    pred_real = denormalize(pred_norm)

    dead = model.dead_neurons(X)

    return {
        "activation": hidden_activation,
        "seed": seed,
        "model": model,
        "epochs": training["epochs"],
        "final_error": training["final_error"],
        "history": training["history"],
        "converged": training["converged"],
        "accuracy": classification_accuracy(Y, pred_norm),
        "mae": mae_real(Y_REAL, pred_norm),
        "pred_norm": pred_norm,
        "pred_real": pred_real,
        "dead_neurons": dead,
        "dead_count": int(np.sum(dead))
    }

# =====
# 5. Серия запусков
# =====

def run_experiments():
    sigmoid_runs = []
    relu_runs = []

    print("\n" + "=" * 84)
    print("ЛАБОРАТОРНАЯ РАБОТА №3 – ВАРИАНТ 10")
    print("Сравнение Сигмоиды и ReLU на скрытом слое")
    print("c0 = -3, c1 = -8, ВСЕ, MLP 2-2-1")
    print("=" * 84)

    print("\nКонфигурация А: Сигмоида на скрытом слое")
    for seed in SEEDS:
        result = run_one("sigmoid", seed)
        sigmoid_runs.append(result)
        status = "ДА" if result["converged"] else "НЕТ"
        print(
            f"seed={seed:2d} | эпох={result['epochs']:5d} | "
            f"Es={result['final_error']:.6f} | "
            f"accuracy={result['accuracy']:.2f} | "
            f"MAE={result['mae']:.6f} | сходимость={status}"
        )

    print("\nКонфигурация В: ReLU на скрытом слое")
    for seed in SEEDS:
        result = run_one("relu", seed)
        relu_runs.append(result)
        status = "ДА" if result["converged"] else "НЕТ"
        dead_text = "".join(str(i + 1) for i, d in enumerate(result["dead_neurons"]) if d)
        if not dead_text:
            dead_text = "нет"
        print(
            f"seed={seed:2d} | эпох={result['epochs']:5d} | "
            f"Es={result['final_error']:.6f} | "
            f"accuracy={result['accuracy']:.2f} | "
            f"MAE={result['mae']:.6f} | сходимость={status} | "
            f"мёртвые ReLU: {dead_text}"
        )

    return sigmoid_runs, relu_runs

# =====
# 6. Представительный запуск
# =====

def choose_representative(runs):

```

```

successful = [r for r in runs if r["converged"]]

if successful:
    epochs = np.array([r["epochs"] for r in successful])
    median_epochs = np.median(epochs)
    return min(successful, key=lambda r: abs(r["epochs"] - median_epochs))

return min(runs, key=lambda r: r["final_error"])

# =====
# 7. Вывод результатов
# =====

def print_summary(sigmoid_runs, relu_runs):
    print("\n" + "=" * 84)
    print("СВОДНАЯ СТАТИСТИКА ПО 5 ЗАПУСКАМ")
    print("=" * 84)

    for title, runs in [("Сигмоида", sigmoid_runs), ("ReLU", relu_runs)]:
        successful = [r for r in runs if r["converged"]]

        print(f"\n{title}:")
        print(f"    Успешная сходимость: {len(successful)} из {len(runs)}")

        if successful:
            epochs = np.array([r["epochs"] for r in successful])
            print(f"    Минимум эпох: {epochs.min()}")
            print(f"    Максимум эпох: {epochs.max()}")
            print(f"    Среднее эпох: {epochs.mean():.2f}")
            print(f"    Разброс эпох: {epochs.max() - epochs.min()}")

            print(f"    Средняя accuracy: {np.mean([r['accuracy'] for r in runs]):.4f}")
            print(f"    Средняя MAE: {np.mean([r['mae'] for r in runs]):.6f}")

            if title == "ReLU":
                print(f"    Запусков с мёртвым нейроном: {sum(r['dead_count'] > 0 for r in runs)}")

def print_prediction_table(title, run):
    print("\n" + title)
    print("-" * 88)
    print(
        f"{'A':>7}{'B':>8}{'Цель':>10}"
        f"{'ŷ norm':>14}{'ŷ real':>14}{'Класс':>12}"
    )
    print("-" * 88)

    for x_real, target, pred_norm, pred_real in zip(
        X_REAL,
        Y_REAL.ravel(),
        run["pred_norm"].ravel(),
        run["pred_real"].ravel()
    ):
        predicted_class = C1 if pred_norm >= 0.5 else C0
        print(
            f"{x_real[0]:7.1f}{x_real[1]:8.1f}{target:10.1f}"
            f"{pred_norm:14.4f}{pred_real:14.4f}{predicted_class:12.1f}"
        )

# =====
# 8. Графики
# =====

def save_current_figure(filename):
    FIG_DIR.mkdir(exist_ok=True)
    plt.savefig(FIG_DIR / filename, dpi=180, bbox_inches="tight")

def plot_convergence(sigmoid_run, relu_run):
    plt.figure(figsize=(10, 6))

    plt.plot(
        np.arange(1, len(sigmoid_run["history"]) + 1),
        sigmoid_run["history"],
        label=f"Сигмоида, seed={sigmoid_run['seed']}"
    )

    plt.plot(
        np.arange(1, len(relu_run["history"]) + 1),
        relu_run["history"],
        label=f"ReLU, seed={relu_run['seed']}"
    )

    plt.axhline(Ee, linestyle="--", label=f"Ee = {Ee}")
    plt.yscale("log")
    plt.xlabel("Номер эпохи")
    plt.ylabel("Суммарная ошибка BCE, Es")
    plt.title("Сходимость: Сигмоида и ReLU — вариант 10")

```

```

plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
save_current_figure("01_convergence.png")

def plot_epochs(sigmoid_runs, relu_runs):
    x = np.arange(len(SEEDS))
    width = 0.36

    sigmoid_epochs = [r["epochs"] for r in sigmoid_runs]
    relu_epochs = [r["epochs"] for r in relu_runs]

    plt.figure(figsize=(10, 6))
    plt.bar(x - width / 2, sigmoid_epochs, width, label="Сигмоида")
    plt.bar(x + width / 2, relu_epochs, width, label="ReLU")

    for i, run in enumerate(sigmoid_runs):
        if not run["converged"]:
            plt.scatter(x[i] - width / 2, run["epochs"], marker="x", s=100, linewidths=2)

    for i, run in enumerate(relu_runs):
        if not run["converged"]:
            plt.scatter(x[i] + width / 2, run["epochs"], marker="x", s=100, linewidths=2)

    if run["dead_count"] > 0:
        plt.text(
            x[i] + width / 2,
            run["epochs"] * 0.96,
            "dead ReLU",
            ha="center",
            va="top",
            rotation=90,
            fontsize=8
        )

    plt.xticks(x, [str(seed) for seed in SEEDS])
    plt.xlabel("Seed")
    plt.ylabel("Количество эпох")
    plt.title("Устойчивость сходимости по 5 инициализациям – вариант 10")
    plt.grid(axis="y", alpha=0.3)
    plt.legend()
    plt.tight_layout()
    save_current_figure("02_epochs_by_seed.png")

def plot_decision_surface(run, title, filename):
    model = run["model"]

    a_values = np.linspace(-10.0, 10.0, 250)
    b_values = np.linspace(-10.0, 10.0, 250)
    aa, bb = np.meshgrid(a_values, b_values)

    grid_real = np.column_stack([aa.ravel(), bb.ravel()])
    grid_norm = normalize(grid_real)
    z = model.predict(grid_norm).reshape(aa.shape)

    plt.figure(figsize=(8, 7))
    contour = plt.contourf(
        aa,
        bb,
        z,
        levels=np.linspace(0.0, 1.0, 21),
        cmap="viridis"
    )
    plt.colorbar(contour, label="Нормализованный выход ŷ")
    plt.contour(aa, bb, z, levels=[0.5], linewidths=2)

    plt.scatter(
        X_REAL[:, 0],
        X_REAL[:, 1],
        c=Y.ravel().astype(int),
        cmap="coolwarm",
        edgecolors="black",
        s=120,
        label="Обучающие примеры"
    )

    plt.xlim(-10, 10)
    plt.ylim(-10, 10)
    plt.xlabel("A")
    plt.ylabel("B")
    plt.title(title)
    plt.grid(True, alpha=0.2)
    plt.legend()
    plt.tight_layout()
    save_current_figure(filename)

def plot_mae(sigmoid_run, relu_run):
    names = ["Сигмоида", "ReLU"]
    values = [sigmoid_run["mae"], relu_run["mae"]]

```

```

plt.figure(figsize=(7, 5))
bars = plt.bar(names, values)
plt.ylabel("Средняя абсолютная ошибка")
plt.title("MAE представительных запусков – вариант 10")
plt.grid(axis="y", alpha=0.3)

for bar, value in zip(bars, values):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height(),
        f"value: {value:.5f}",
        ha="center",
        va="bottom"
    )

plt.tight_layout()
save_current_figure("05_mae.png")

# =====
# 9. Режим функционирования
# =====

def infer_one(model, a, b):
    x_real = np.array([[a, b]], dtype=float)
    x_norm = normalize(x_real)

    y_norm = float(model.predict(x_norm)[0, 0])
    y_real = float(denormalize(y_norm))
    nearest_class = C1 if y_norm >= 0.5 else C0

    return y_norm, y_real, nearest_class

def demonstrate_functioning(sigmoid_model, relu_model):
    print("\n" + "=" * 84)
    print("РЕЖИМ ФУНКЦИОНИРОВАНИЯ")
    print("=" * 84)

    examples = [
        (-3.0, -3.0),
        (-3.0, -8.0),
        (-8.0, -3.0),
        (-8.0, -8.0),
        (-5.5, -5.5),
        (-2.0, -7.0),
        (-9.0, -4.0)
    ]

    for a, b in examples:
        s_norm, s_real, s_class = infer_one(sigmoid_model, a, b)
        r_norm, r_real, r_class = infer_one(relu_model, a, b)

        print(f"\nA={a:.2f}, B={b:.2f}")
        print(
            f"Сигмоида:  $\hat{y}=\{s\_norm:.4f\}$ , "
            f" $y\_real=\{s\_real:.4f\}$ , класс= $\{s\_class:.1f\}$ "
        )
        print(
            f"ReLU:  $\hat{y}=\{r\_norm:.4f\}$ , "
            f" $y\_real=\{r\_real:.4f\}$ , класс= $\{r\_class:.1f\}$ "
        )

def interactive_mode(sigmoid_model, relu_model):
    print("\nВведите А и В из [-10; 10]. Пустая строка – выход.")

    while True:
        raw = input("\nA B: ").strip()
        if not raw:
            break

        try:
            a, b = map(float, raw.split())
        except ValueError:
            print("Ошибка: введите два числа через пробел.")
            continue

        if not (-10 <= a <= 10 and -10 <= b <= 10):
            print("Ошибка: числа должны быть в диапазоне [-10; 10].")
            continue

        s_norm, s_real, s_class = infer_one(sigmoid_model, a, b)
        r_norm, r_real, r_class = infer_one(relu_model, a, b)

        print(
            f"Сигмоида:  $\hat{y}=\{s\_norm:.4f\}$ , "
            f" $y\_real=\{s\_real:.4f\}$ , класс= $\{s\_class:.1f\}$ "
        )
        print(
            f"ReLU:  $\hat{y}=\{r\_norm:.4f\}$ , "
            f" $y\_real=\{r\_real:.4f\}$ , класс= $\{r\_class:.1f\}$ "
        )

```

```

    )

# =====
# 10. Главная программа
# =====

def main():
    sigmoid_runs, relu_runs = run_experiments()

    sigmoid_rep = choose_representative(sigmoid_runs)
    relu_rep = choose_representative(relu_runs)

    print_summary(sigmoid_runs, relu_runs)

    print_prediction_table(
        f"Представительный запуск Сигмоида (seed={sigmoid_rep['seed']})",
        sigmoid_rep
    )
    print_prediction_table(
        f"Представительный запуск ReLU (seed={relu_rep['seed']})",
        relu_rep
    )

    demonstrate_functioning(sigmoid_rep["model"], relu_rep["model"])

    plot_convergence(sigmoid_rep, relu_rep)
    plot_epochs(sigmoid_runs, relu_runs)
    plot_decision_surface(
        sigmoid_rep,
        f"Разделяющая поверхность: Сигмоида, seed={sigmoid_rep['seed']}",
        "03_surface_sigmoid.png"
    )
    plot_decision_surface(
        relu_rep,
        f"Разделяющая поверхность: ReLU, seed={relu_rep['seed']}",
        "04_surface_relu.png"
    )
    plot_mae(sigmoid_rep, relu_rep)

    plt.show()

    if sys.stdin.isatty():
        interactive_mode(sigmoid_rep["model"], relu_rep["model"])

if __name__ == "__main__":
    main()

```

Результат программы:

Конфигурация А — Сигмоида на скрытом слое

Seed	Эпохи	Es	Accuracy	MAE	Сходимость
1	5365	0.029992	1.00	0.037342	ДА
4	6305	0.029991	1.00	0.037341	ДА
5	6040	0.029999	1.00	0.037351	ДА
6	5627	0.029998	1.00	0.037349	ДА
7	5916	0.029991	1.00	0.037341	ДА

Конфигурация В — ReLU на скрытом слое

Seed	Эпохи	Es	Accuracy	MAE	Сходимость	Мёртвые ReLU
1	859	0.029966	1.00	0.037270	ДА	нет
4	1023	0.029979	1.00	0.037207	ДА	нет
5	20000	1.909576	0.75	1.665560	НЕТ	нейрон 1
6	800	0.029967	1.00	0.037272	ДА	нет
7	923	0.029925	1.00	0.037219	ДА	нет

Сводная статистика

Конфигурация	Успешно	Мин. эпох	Макс. эпох	Среднее эпох	Разброс	Средняя MAE
Сигмоида	5/5	5365	6305	5850,60	940	0,037345
ReLU	4/5	800	1023	901,25	223	0,362906*

* Средняя MAE ReLU по всем пяти запускам увеличена из-за неуспешного запуска seed=5 с мёртвым нейроном.

Представительный запуск Сигмоиды (seed=7)

А	В	Цель	$\hat{y}$ norm	$\hat{y}$ real	Класс
-3.0	-3.0	-3.0	0.0061	-3.0304	-3.0
-3.0	-8.0	-8.0	0.9933	-7.9664	-8.0
-8.0	-3.0	-8.0	0.9933	-7.9664	-8.0
-8.0	-8.0	-3.0	0.0104	-3.0518	-3.0

Представительный запуск ReLU (seed=1)

А	В	Цель	$\hat{y}$ norm	$\hat{y}$ real	Класс
-3.0	-3.0	-3.0	0.0031	-3.0157	-3.0
-3.0	-8.0	-8.0	0.9882	-7.9411	-8.0
-8.0	-3.0	-8.0	0.9882	-7.9411	-8.0
-8.0	-8.0	-3.0	0.0031	-3.0157	-3.0

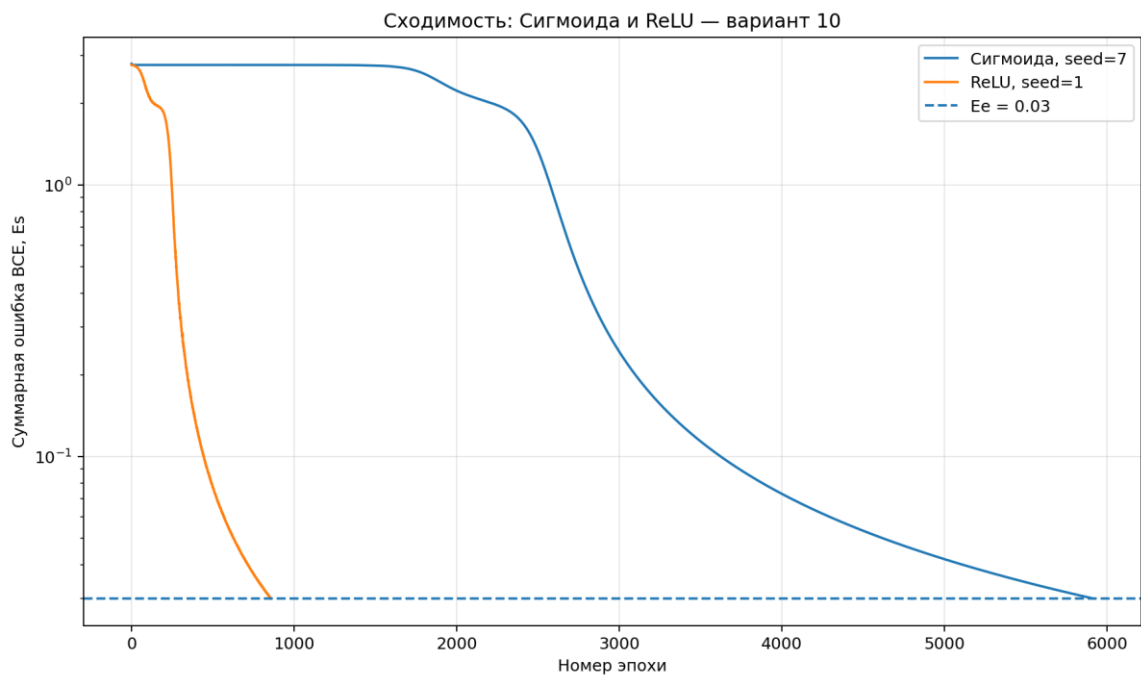


Рисунок 1 — График сходимости Сигмоиды и ReLU

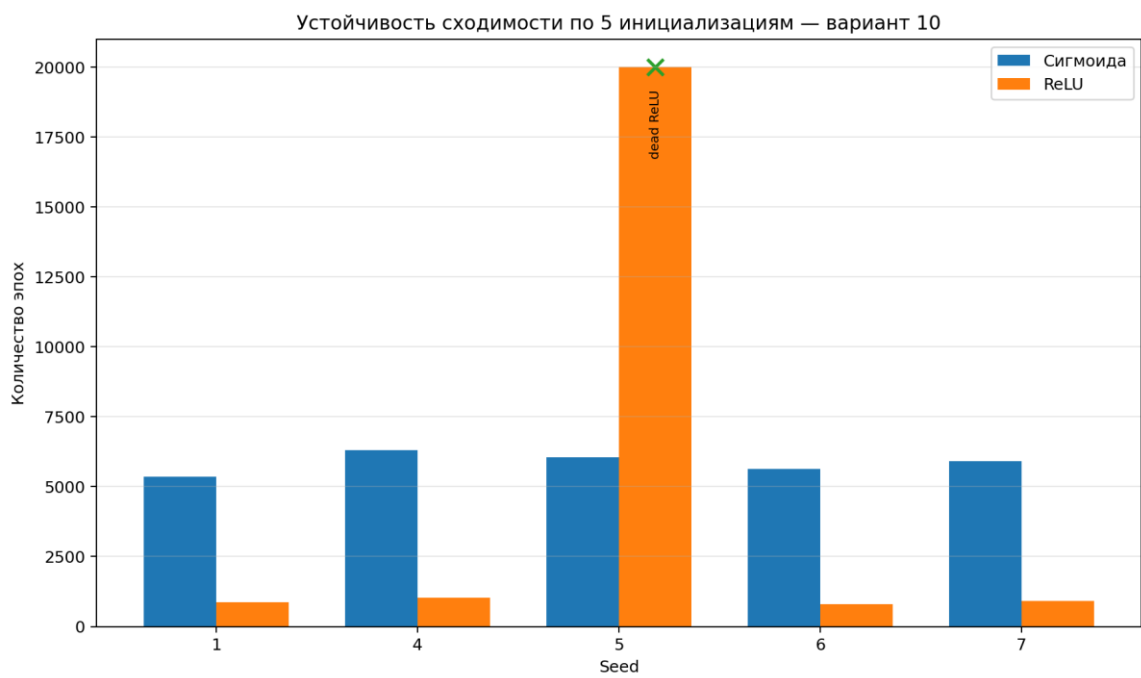


Рисунок 2 — Устойчивость сходимости по 5 инициализациям

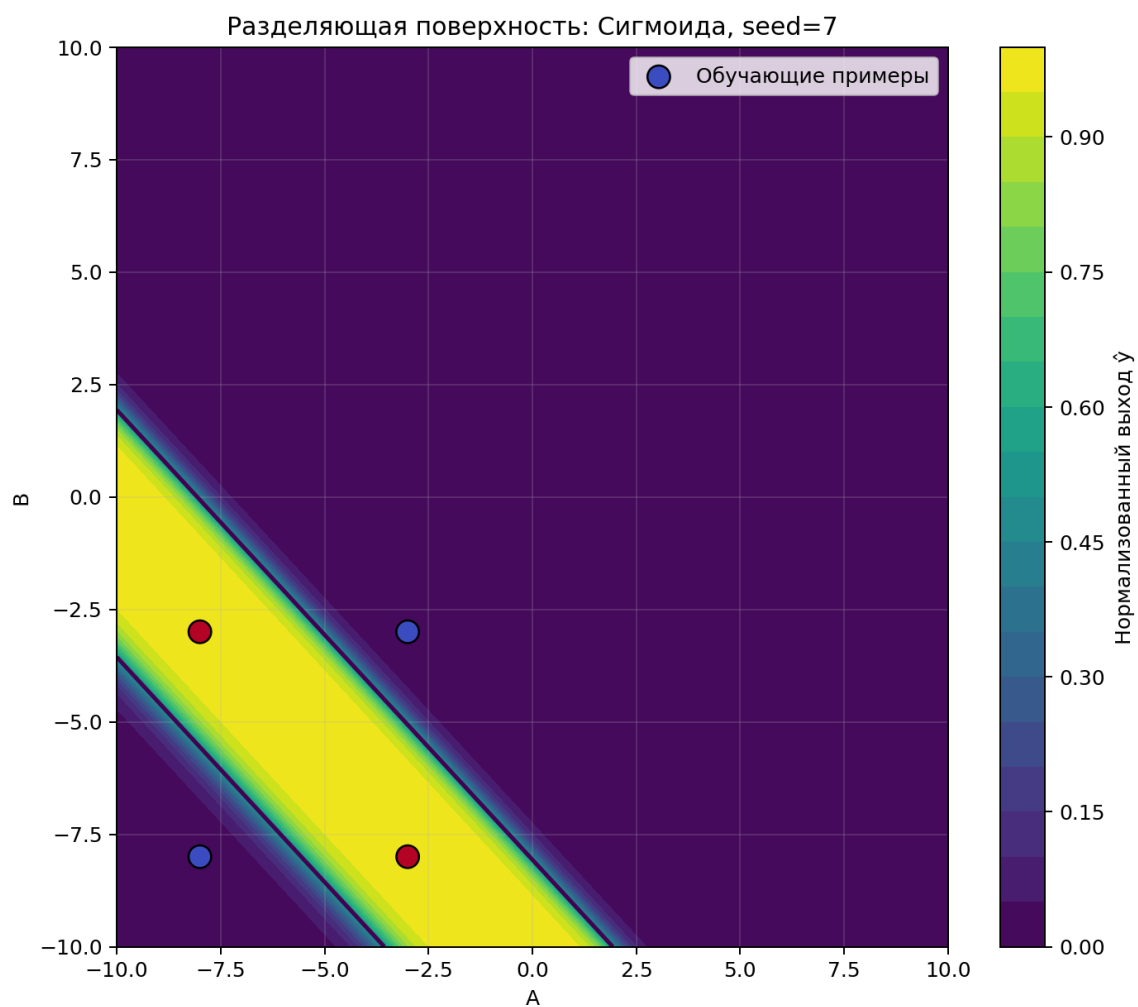


Рисунок 3 — Разделяющая поверхность для Сигмоиды

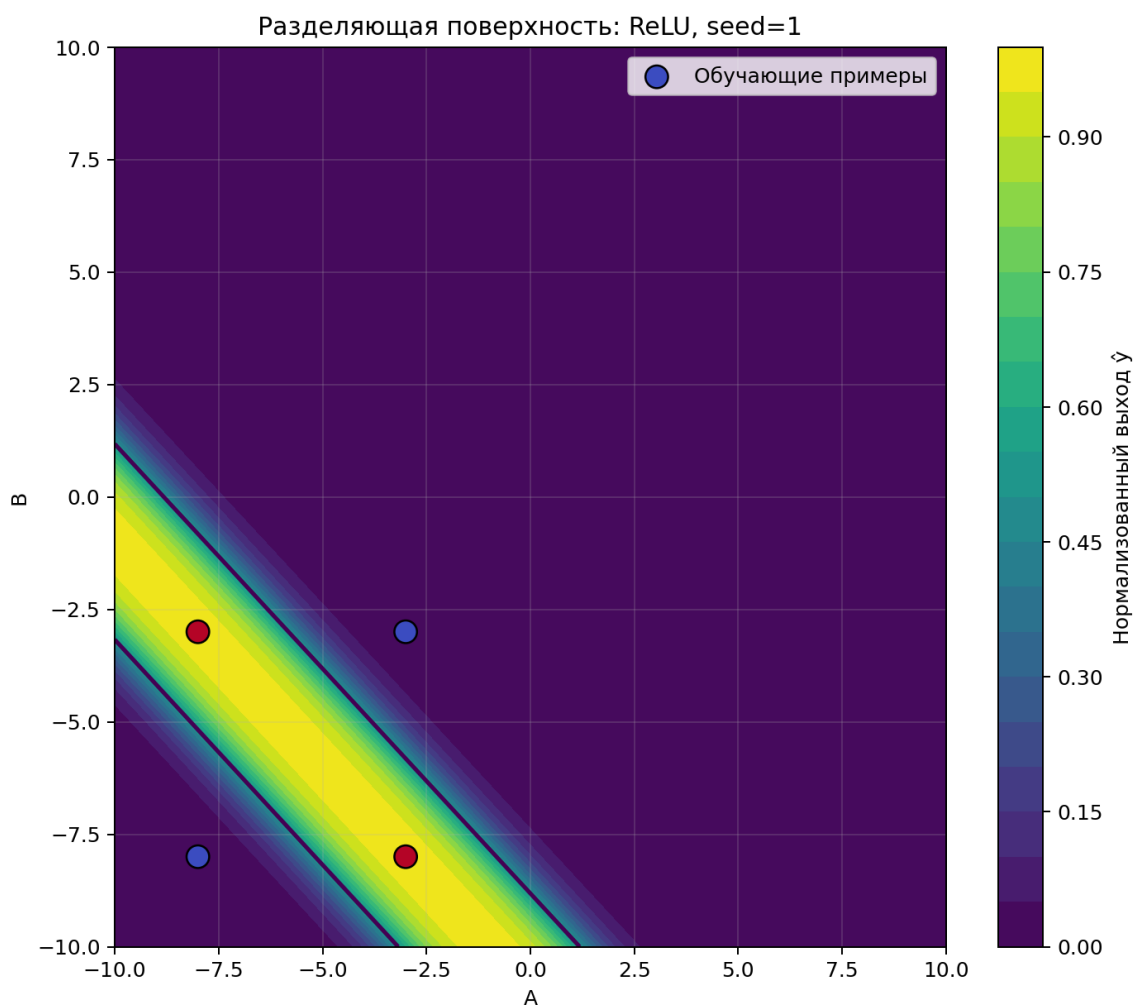
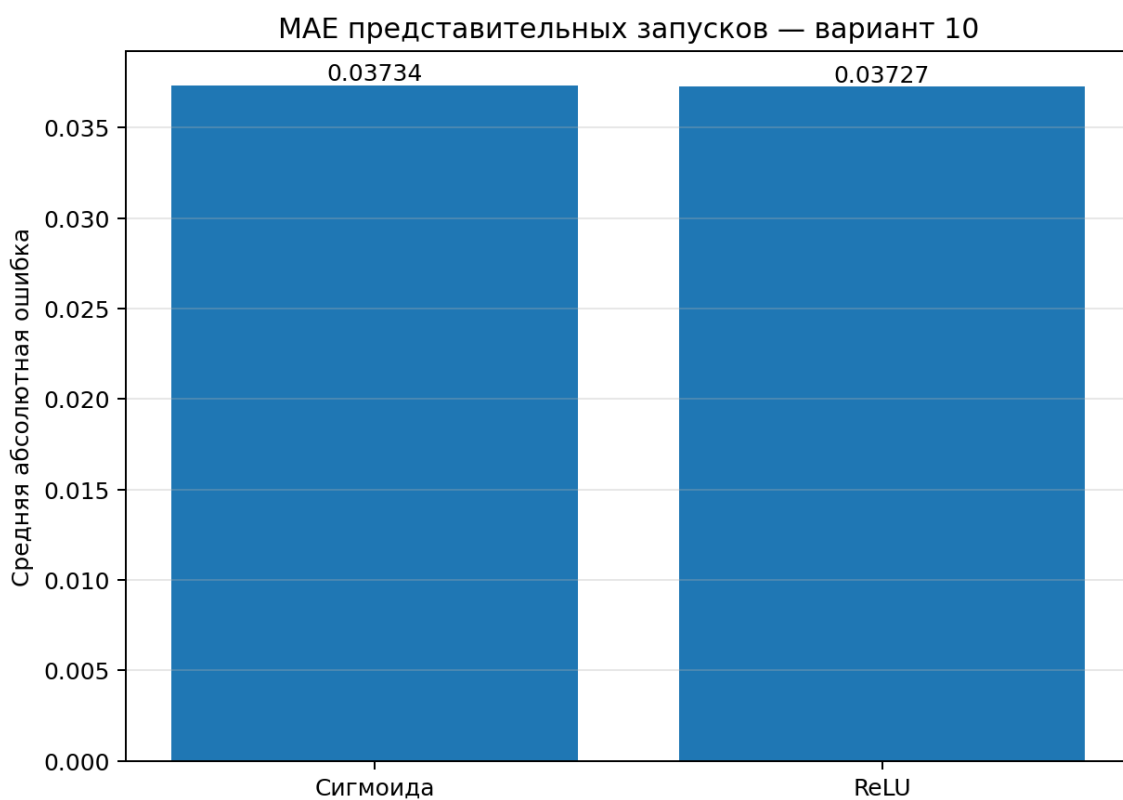


Рисунок 4 — Разделяющая поверхность для ReLU



Режим функционирования

Ниже приведены результаты для четырёх обучающих примеров и трёх дополнительных пар входов. Для каждой конфигурации выводится нормализованный результат, значение в исходной шкале и ближайший класс.

A	B	Sig $\hat{y}$	Sig real	Sig класс	ReLU $\hat{y}$	ReLU real	ReLU класс
-3.00	-3.00	0.0061	-3.0304	-3.0	0.0031	-3.0157	-3.0
-3.00	-8.00	0.9933	-7.9664	-8.0	0.9882	-7.9411	-8.0
-8.00	-3.00	0.9933	-7.9664	-8.0	0.9882	-7.9411	-8.0
-8.00	-8.00	0.0104	-3.0518	-3.0	0.0031	-3.0157	-3.0
-5.50	-5.50	0.9933	-7.9664	-8.0	0.9882	-7.9411	-8.0
-2.00	-7.00	0.9652	-7.8258	-8.0	0.5927	-5.9633	-8.0
-9.00	-4.00	0.8376	-7.1881	-8.0	0.5890	-5.9448	-8.0

Краткий анализ результатов

1. Сходимость. Сигмоида успешно сошлась во всех 5 запусках. ReLU успешно сошлась в 4 из 5 запусков, но успешные запуски ReLU были значительно быстрее.
2. Скорость. Среднее число эпох для Сигмоиды составило 5850,60, а для успешных запусков ReLU — 901,25. Поэтому ReLU в данном эксперименте сходилась примерно в 6,5 раза быстрее.
3. Устойчивость. У успешных запусков ReLU разброс составил 223 эпохи против 940 у Сигмоиды, однако один запуск ReLU не сошёлся из-за мёртвого нейрона.
4. Разделяющие поверхности. Обе конфигурации правильно формируют нелинейную границу XOR. Поверхность Сигмоиды выглядит более гладкой, а ReLU — более кусочно-линейной.
5. Качество. В представительных успешных запусках обе сети дали ассигасу = 1,00 и близкую MAE. ReLU показывает высокое качество при успешной инициализации, но чувствительнее к проблеме мёртвых нейронов.

Ответы на вопросы для вывода

1. **Какая из активационных функций обеспечила более быструю сходимость? Более устойчива?** ReLU обеспечила существенно более быструю сходимость: успешные запуски занимали в среднем 901,25 эпохи против 5850,60 у Сигмоиды. По разбросу успешных запусков ReLU также была стабильнее, однако один запуск не сошёлся из-за мёртвого нейрона. Сигмоида сошлась во всех 5 случаях.
2. **Как отличается форма разделяющей поверхности?** Обе конфигурации формируют корректную нелинейную границу XOR. У Сигмоиды переходы более плавные, а ReLU формирует более прямолинейные, кусочно-линейные участки, что соответствует свойствам этих функций активации.
3. **Почему ReLU часто предпочитают Сигмоиде в глубоких сетях? Подтверждается ли это на XOR?** ReLU не насыщается на положительной области и обычно передаёт более сильный градиент, поэтому обучение может

идти быстрее. В данном эксперименте это подтвердилось: успешные запуски ReLU сходились значительно быстрее Сигмоиды.

4. Изменилась ли способность сети решать XOR при переходе на ReLU? Нет. Обе активационные функции позволяют решить XOR при наличии скрытого слоя. Представительные успешные экземпляры обеих конфигураций достигли ассигасы = 1,00.

5. Есть ли случаи «мёртвых нейронов» при ReLU? Да. При seed=5 один скрытый ReLU-нейрон стал мёртвым. Этот запуск не достиг критерия остановки за 20000 эпох и получил ассигасу = 0,75, что показало влияние проблемы мёртвых нейронов на обучение.

6. Как выбор активационной функции соотносится с необходимостью многослойной архитектуры для XOR? Выбор активации влияет на скорость и устойчивость обучения, но не отменяет необходимость скрытого слоя. XOR остаётся линейно неразделимой задачей и требует многослойной архитектуры.

7. Что показывает визуализация разделяющей поверхности? Обе поверхности правильно разделяют четыре XOR-точки: две комбинации класса $c1 = -8$ попадают в область высокого выхода, а комбинации класса $c0 = -3$ — в область низкого выхода. У ReLU граница визуально более кусочно-линейная.

Вывод:

В ходе лабораторной работы был исследован многослойный персептрон 2-2-1 для задачи XOR варианта 10 с классами $c0 = -3$ и $c1 = -8$. Были сравнены Сигмоида и ReLU на скрытом слое при использовании ВСЕ и Сигмоиды на выходе. Сигмоида показала максимальную надёжность и сошлась во всех пяти запусках. ReLU в четырёх успешных запусках обучалась примерно в 6,5 раза быстрее и имела меньший разброс числа эпох, однако при одной инициализации возник мёртвый нейрон, из-за чего сеть не достигла критерия остановки. Обе конфигурации при успешном обучении корректно решают XOR, что подтверждает определяющую роль многослойной архитектуры, а выбор активационной функции главным образом влияет на скорость и устойчивость обучения.